# Flow of Control

16 October 2026 06:42

**Selection:**

- **if block**
- **if … else block**
- **if … elif… else block**

**Syntax:**

```
if <condition> :
   #statement(s)


if <condition> :
   #statement(s)
else:
   #statement(s)


if <condition> :
   #statement(s)
elif <condition>:
   #statement(s)
…
else:
   #statement(s)
```

**Display the appropriate message as per the color of signal at the road crossing.**

```
signal = input('Enter the color: ')
if signal == "Red" or signal == "RED":
    print('STOP')
elif signal == "Orange" or signal == "ORANGE":
    print("Be slow")
elif signal == "Green" or signal == "GREEN":
    print("Go!")
else:
    print('Invalid input!')
```

**Output:**

```
PS C:\Users\bluej\Documents\Stuc
xe c:/Users/bluej/Documents/Stuc
nal.py
Enter the color: Red
STOP
PS C:\Users\bluej\Documents\Stuc
xe c:/Users/bluej/Documents/Stuc
nal.py
Enter the color: Green
Go!
PS C:\Users\bluej\Documents\Stuc
xe c:/Users/bluej/Documents/Stuc
nal.py
Enter the color: Yellow
Invalid input!
PS C:\Users\bluej\Documents\Stuc
```

**Another simple selection in Python:**

**match case statement**

**Syntax:**

```
match <expression>:
   case <match value1>:
      #Statement(s)

   case <match value1>:
      #Statement(s)

   case <match value1>:
      #Statement(s)
   …
   case _ : #default case, when all the case fails then it executes
      #statement(s)
```

```
prompt = "Give your operator: +, -, *, /, \n"
prompt += "//(lower division), %(remainder), **(exponential)\n"
oper = input(prompt)
n1 = int(input('Enter first number: '))
n2 = int(input('Enter second number: '))
match oper:
    case '+':
        print(f"sum = {n1+n2}")
    case '-':
        print(f"diff = {n1-n2}")
    case '*':
        print(f"product = {n1*n2}")
    case '/':
        print(f"divison = {n1/n2}")
    case "//":
        print(f"Quotient = {n1//n2}")
    case "%":
        print(f"remainder = {n1%n2}")
    case "**":
```

```
        print(f"{n1} to the power {n2} = {n1**n2}")
    case _:
        print("Wrong option selected")
```

**Output:**

```
Give your operator: +, -, *, /,
//(lower division), %(remainder), **(exponential)
//
Enter first number: 10
Enter second number: 4
Quotient = 2
PS C:\Users\bluej\Documents\Students\KapishKhemkaLMB2
orGit\Coding\PythonProjects> & c:\python314\python.ex
Users/bluej/Documents/Students/KapishKhemkaLMB2628/Fo
Coding/PythonProjects/Chapter1/calculator.py
Give your operator: +, -, *, /,
//(lower division), %(remainder), **(exponential)
add
Enter first number: 41
Enter second number: 43
Wrong option selected
```

**Indentation:** In most programming languages, the statements within a block are put inside curly brackets. However, Python uses indentation for block as well as for nested block structures. Leading white spaces (spaces and tabs) at the beginning of the statement is called indentation. In Python, the same level of indentation associate statements into a single block of code. The interpreter checks indentation levels very strictly and throws up syntax error if indentation is not correct. It is a common practice to use single tab for each level of indentation.

**Repetition (Loop)**

**i. while loop**

**Syntax:**

```
while <condition>:
    #statement(s)
```

```
start = 1
end = 10
while start <= end:
    print(start, end = ' ')
    start += 1
```

**Output:**

**1 2 3 4 5 6 7 8 9 10**

**2. The 'for' loop**

```
>>> list(range(1,10))
[1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list(range(1,10,2))
[1, 3, 5, 7, 9]
>>> list(range(1,10,-1))
```

range(start, stop, step (jump))

it generates a sequence of valid indexes.

```
>>> list(range(1,10,2))
[1, 3, 5, 7, 9]
>>> list(range(1,10,-1))
[]
>>> list(range(10,1,-1))
[10, 9, 8, 7, 6, 5, 4, 3, 2]
```

it generates a sequence of valid indexes.

[if start > stop, then step must be -ve
if start < stop, the step must be +ve]

'step' cannot be zero otherwise ValueError will be generated.

If the start, stop values are not given logically it will return empty list [], and hence loop will not execute.

**Syntax of the 'for' loop:**

```
for <control-variable> in <sequence/items in range>:
   #statements inside body of the for loop>
```

```
start = 1
end = 11
for i in range(start, end):
    print(i, end=' ')
print()
# If I have to reverse the loop.
for i in range(end-1, start-1, -1):
    print(i, end=' ')
```

**Output:**

```
1 2 3 4 5 6 7 8 9 10
10 9 8 7 6 5 4 3 2 1
```

**Nested loop:**
**A loop present inside a loop, or the inner loop or nested loop will complete for each value of outer loop. Example, clock.**